

OpenGL - Bases du rendu temps réel

Adam HAMOUCHE

Avril 2026

1 Introduction

Ce document présente les bases de la programmation graphique et du rendu en temps réel avec OpenGL moderne. L'objectif est de comprendre le pipeline de rendu ainsi que les différentes étapes permettant de transformer des données géométriques en image affichée à l'écran.

2 Pipeline de rendu

Le pipeline de rendu est une suite d'étapes exécutées majoritairement sur le GPU permettant de transformer des données 3D en pixels affichés à l'écran.

Les principales étapes sont :

- **Vertex Processing** : Les sommets (vertices) sont transformés via le vertex shader. On applique généralement les matrices Model, View et Projection pour passer de l'espace local à l'espace clip.
- **Primitive Assembly** : Les sommets sont assemblés en primitives (triangles, lignes, etc.).
- **Rasterization** : Les primitives sont converties en fragments (candidats pixels).
- **Fragment Processing** : Chaque fragment est traité par le fragment shader, qui calcule sa couleur et éventuellement d'autres propriétés (profondeur, transparence).
- **Tests et Blending** : Tests comme le depth test (Z-buffer) ou le stencil test, puis mélange des couleurs (blending).
- **Framebuffer** : Les fragments finaux sont écrits dans le framebuffer pour affichage.

3 Processus de Rasterization

Le processus de rasterization consiste à transformer des primitives géométriques en fragments affichables à l'écran.

Pour cela, chaque sommet passe par plusieurs transformations :

- **Model Matrix** : transforme les coordonnées locales en coordonnées monde (translation, rotation, scale).
- **View Matrix** : transforme les coordonnées monde en coordonnées caméra (espace vue).
- **Projection Matrix** : projette la scène 3D en espace 2D (projection perspective ou orthographique).

Ces transformations donnent des coordonnées en **espace clip**. Après division par la composante w , on obtient des coordonnées en **Normalized Device Coordinates (NDC)** dans l'intervalle $[-1, 1]$.

3.1 Viewport Transform

Le **viewport transform** convertit les coordonnées NDC en coordonnées écran.

- Les coordonnées $[-1, 1]$ sont transformées en coordonnées pixels.
- Cette transformation dépend de la taille de la fenêtre (width, height).

Exemple :

- $(-1, -1) \rightarrow$ coin bas gauche de l'écran
- $(1, 1) \rightarrow$ coin haut droit

Cela permet de positionner correctement les fragments dans le framebuffer.

3.2 Framebuffer

Le **framebuffer** est une zone mémoire contenant les données finales à afficher à l'écran.

Il contient généralement plusieurs buffers :

- **Color Buffer** : stocke la couleur des pixels
- **Depth Buffer (Z-buffer)** : stocke la profondeur pour gérer l'occlusion
- **Stencil Buffer** : utilisé pour des tests avancés (masques, effets)

Lorsque tous les fragments sont traités, leur résultat est écrit dans le framebuffer. Celui-ci est ensuite affiché à l'écran (swap buffers).

4 Shader Programming

Les shaders sont des programmes exécutés sur le GPU, permettant de définir le comportement des différentes étapes du pipeline graphique.

En OpenGL moderne, les principales étapes programmables sont :

- **Vertex Shader** : exécuté pour chaque vertex, il transforme les positions (ex: application de la matrice MVP).
- **Fragment Shader** : exécuté pour chaque fragment, il calcule la couleur finale du pixel.

Les shaders sont écrits en GLSL (OpenGL Shading Language), un langage proche du C.

Les données sont transmises entre shaders via des variables **in** et **out**. Les sorties du vertex shader sont interpolées puis passées en entrée du fragment shader. Chaque attribut est associé à un index via le mot-clé **layout**.

Les shaders sont compilés à l'exécution, puis liés dans un **shader program** (via `glCreateProgram`). Ce programme est ensuite activé avec `glUseProgram` pour être utilisé lors du rendu.

5 Buffers

En OpenGL moderne, les données (vertices, indices, etc.) sont stockées dans des buffers envoyés au GPU. Cela permet un accès efficace et évite les échanges coûteux avec le CPU.

5.1 VBO (Vertex Buffer Object)

Le VBO contient les données des vertices (positions, normales, UV, etc.).

Il est créé avec `glGenBuffers` puis rempli avec `glBufferData`. Un VBO peut contenir les données d'un ou plusieurs objets afin d'optimiser les performances.

5.2 EBO (Element Buffer Object)

L'EBO contient des indices qui référencent les vertices du VBO.

Il permet de réutiliser les mêmes vertices pour plusieurs triangles, évitant ainsi la duplication des données en mémoire et réduisant le coût de transfert vers le GPU.

5.3 VAO (Vertex Array Object)

Le VAO ne contient pas les données, mais la configuration permettant de les interpréter.

Il stocke :

- la description des attributs de vertex (`glVertexAttribPointer`)

- les attributs activés
- les VBO associés aux attributs
- l'EBO lié

Le VAO agit comme une "fiche de configuration" : il décrit comment lire les données du VBO (taille, type, stride, offset).

Une fois configuré, il suffit de bind le VAO pour restaurer automatiquement tout l'état nécessaire au rendu.

Lors d'un appel à `glDrawElements`, OpenGL utilise le VAO pour :

- lire correctement les données du VBO
- utiliser les indices de l'EBO pour assembler les primitives

6 Texturing

Le texturing en OpenGL consiste à appliquer une image (texture) sur un objet 3D afin d'ajouter du détail visuel sans augmenter la complexité géométrique. Pour cela, chaque vertex possède des coordonnées UV (comprises entre 0 et 1) qui indiquent comment la texture est projetée sur le mesh. Lors du rendu, le fragment shader utilise ces coordonnées pour échantillonner la texture via la fonction `texture()`, ce qui permet de récupérer la couleur correspondante. Afin d'améliorer la qualité et les performances, différentes techniques sont utilisées : le filtrage (nearest ou linear) pour gérer l'interpolation des pixels, et les mipmaps, qui sont des versions réduites de la texture permettant d'optimiser l'affichage des objets éloignés et de limiter l'aliasing. En OpenGL, une texture est créée et configurée via des appels comme `glGenTextures`, `glBindTexture` et `glTexImage2D`, puis liée au shader pour être utilisée lors du rendu.

7 Lighting

Le lighting en OpenGL consiste à simuler l'interaction entre la lumière et les surfaces afin de déterminer la couleur finale des objets à l'écran. En OpenGL moderne, ces calculs sont réalisés dans les shaders, généralement dans le fragment shader.

7.1 Modèle de Phong (ADS)

Le modèle de lighting le plus classique est le modèle de Phong, basé sur trois composantes :

- **Ambient** : lumière globale et uniforme, indépendante de la position ou de l'orientation des objets.

- **Diffuse** : dépend de l'angle entre la normale de la surface et la direction de la lumière. Plus la surface est orientée vers la lumière, plus elle est éclairée.
- **Specular** : correspond au reflet brillant (highlight). Elle dépend de la direction de la caméra et du paramètre de brillance du matériau (shininess).

7.2 Types de lumières

On distingue principalement trois types de lumières :

- **Directional Light** : lumière de direction constante, sans position (ex : soleil).
- **Point Light** : lumière émise dans toutes les directions à partir d'une position donnée, avec atténuation en fonction de la distance.
- **Spot Light** : lumière en forme de cône, définie par une position, une direction et un angle d'ouverture.

7.3 Rôle des normales

Le calcul de la lumière repose sur les normales des surfaces. Celles-ci doivent être normalisées et correctement transformées dans l'espace utilisé pour les calculs (souvent l'espace monde).

En cas de transformations non uniformes (scale), les normales doivent être corrigées à l'aide de la **normal matrix**, définie comme la transposée de l'inverse de la matrice modèle.

7.4 Matériaux

Chaque objet possède un matériau qui influence sa manière de réagir à la lumière. Un matériau est généralement défini par :

- une composante ambiante
- une composante diffuse
- une composante spéculaire
- un coefficient de brillance (shininess)

7.5 Implémentation

En OpenGL moderne, toutes les informations nécessaires au lighting (lumières et matériaux) sont envoyées au shader via des variables uniformes. Pour des scènes plus complexes, il est possible d'utiliser des Uniform Buffer Objects (UBO) afin d'optimiser le transfert et le partage des données entre plusieurs shaders.

8 Culling et Clipping

Le culling et le clipping regroupent différentes techniques d'optimisation visant à ne rendre que les éléments visibles à l'écran, afin de réduire le coût de calcul et améliorer les performances.

8.1 Backface Culling

Le backface culling consiste à ne pas rendre les faces orientées à l'opposé de la caméra.

Il repose sur l'orientation des normales : si une face est tournée vers l'arrière, elle ne peut pas être visible et est donc ignorée. Cela permet de réduire le nombre de fragments à traiter.

8.2 Frustum Culling

Le frustum culling consiste à ne pas envoyer au GPU les objets situés en dehors du champ de vision de la caméra (frustum).

Cette technique est généralement implémentée côté CPU, car OpenGL ne la gère pas automatiquement. Elle permet de réduire le nombre de draw calls et la charge GPU.

8.3 Occlusion Culling

L'occlusion culling consiste à ne pas rendre les objets cachés par d'autres objets.

Contrairement au frustum culling, ces objets sont bien dans le champ de la caméra, mais ne sont pas visibles car ils sont masqués. Cette technique permet de limiter l'overdraw (rendu de pixels inutiles).

8.4 Scissor Test

Le scissor test permet de restreindre le rendu à une zone rectangulaire de l'écran.

Les fragments en dehors de cette zone sont ignorés lors de l'écriture dans le framebuffer. Cela est utile pour limiter le rendu à une région spécifique (interface utilisateur, split-screen, etc.).

8.5 Clipping

Le clipping consiste à découper les primitives (triangles) qui intersectent le frustum de la caméra afin de ne conserver que la partie visible.

Contrairement au culling, qui supprime entièrement des objets ou des faces, le clipping opère au niveau des primitives partiellement visibles.

9 Instanced rendering

Tout comme dans Unreal avec le `UInstancedStaticMeshComponent` on a la possibilité de faire du rendu d'un grand nombre d'instance optimisé avec `DrawElementsInstanced(GLEnum.Triangles, (uint)(mesh[i].IndexCount), GLEnum.UnsignedInt, null, ((uint)num));` qui regroupe plusieurs instances dans un seul et même draw call.